Auto-Par: A Compiler-Inspired Source-to-Source Parallelization Tool for Affine Loop Nests Using OpenMP

HOSSAMALDEEN ELGHAZALY[1], FADY FAROUK[1], YOUSSEF MAGED[1], RAMEZ THABET[1] AND OMAR MAHMOUD[1]

[1] School of Information Technology and Computer Science (ITCS), Nile University (NU), Giza, Egypt

CORRESPONDING AUTHOR: H. ELGHAZALY (e-mail: h.rabie2272@nu.edu.eg)

ABSTRACT Writing parallel code by hand is a difficult and error-prone task. Developers must deeply understand how data flows between loop iterations, identify potential race conditions, and carefully apply synchronization primitives a burden that causes most programmers to leave the performance of modern multicore processors largely untapped. This paper presents *Auto-Par*, a lightweight, compiler-inspired tool that reads sequential C code and automatically produces a parallelized version by inserting the appropriate OpenMP directives with no manual effort from the programmer.Auto-Par implements a seven-step pipeline that first identifies analyzable code regions using `#pragma scop` markers, then applies an eight-stage dependency analysis engine to determine whether each loop can be safely parallelized, reduced, or must be left unchanged. For every loop the tool makes a decision and explains it: a human-readable report records the outcome, the reason, and the exact line number, making the process fully transparent.We evaluated the tool against three scientific benchmarks a vector arithmetic suite, a 2D Jacobi heat diffusion solver, and a matrix multiplication kernel running on an 8-core processor. The tool correctly classified 100% of the 10 analyzed loops and achieved a parallelization rate of 80%. Peak measured speedups reached **3.17**× for heat diffusion and **2.46**× for matrix multiplication, with parallel efficiency of 85.5% at two threads for the heat solver. The vector addition benchmark correctly exhibited memory-bandwidth-limited behavior, confirming that the tool's classifications are accurate and that observed performance bounds are hardware-imposed rather than tool errors. These results demonstrate that a transparent, regex-based analysis pipeline is sufficient to automatically parallelize affine scientific loop nests while maintaining strict numerical correctness.

INDEX TERMS Automatic Parallelization, OpenMP, Dependency Analysis, Source-to-Source Compilation, SCoP, Loop Parallelization, Speedup.

I. INTRODUCTION

MODERN processors ship with more cores than most software ever uses. A typical desktop machine today has 8 to 16 physical cores, yet the majority of scientific and engineering applications continue to run on a single core, executing one instruction at a time. The gap between available hardware parallelism and the parallelism actually exploited by software represents one of the most persistent inefficiencies in computing.

The root cause is not hardware it is the difficulty of writing parallel code. To parallelize a loop with OpenMP, a developer must prove that no iteration depends on the result of another, check that no two threads write to the same memory location, identify reduction patterns, handle pointer aliasing, and choose an appropriate scheduling policy. A single oversight can produce results that are silently wrong or crashes that are hard to reproduce. As a result, most developers write sequential code and accept the performance penalty.

Automatic parallelization addresses this by having a tool perform the analysis on behalf of the programmer. The idea has existed since the 1980s, and mature tools such as GCC's auto-parallelizer and LLVM's Polly implement it at the compiler level. However, these tools operate deep inside the compiler backend, making their decisions invisible and their output difficult to inspect or trust. For developers who want to understand *why* a loop was or was not parallelized, these tools offer no answer.

This paper presents *Auto-Par*, a source-to-source translation tool that takes sequential C code as input and produces parallelized C code as output, with `#pragma omp` directives inserted in the right places and an explanation provided for every decision. The tool is built entirely in Python, requires no compiler infrastructure beyond GCC,

and is designed to be readable and understandable not just correct.

The main contributions of this work are:

1) **A working automatic parallelizer:** Auto-Par implements a complete seven-step pipeline from sequential C input to OpenMP-annotated output, driven by an eight-stage dependency analysis engine that checks for dynamic memory allocation, recursion, unknown function calls, pointer aliasing, reduction patterns, loop-carried dependencies, and write conflicts in that priority order.
2) **Broad pattern support:** The tool handles affine `for`-loops, counter-based `while`-loops (converted internally to `for`-form), scalar reduction patterns (`+=`, `*=`), and pointer arithmetic of the form `*(ptr + i)` with arbitrary whitespace all correctly classified and transformed.
3) **Full transparency:** For every analyzed loop the tool prints a labeled decision (PARALLEL, REDUCTION, SKIPPED, or UNSAFE) together with the reason. An optional `report.txt` records the complete analysis across all files in a single combined document, making the tool suitable for educational and auditing purposes.
4) **Empirical evaluation:** We evaluate Auto-Par across three scientific benchmarks on an 8-core processor, measuring speedup, parallel efficiency, synchronization overhead, and Amdahl's Law compliance, supported by ten publication-quality figures generated automatically by the benchmarking harness.

The remainder of this paper is organized as follows. Section II reviews related work in automatic parallelization. Section III describes the design of Auto-Par and its analysis pipeline. Section IV presents the experimental evaluation and performance results. Section V concludes and outlines directions for future work.

II. RELATED WORK

A. Theoretical Foundations

The mathematical basis for automatic parallelization was established by Midkiff [1], whose comprehensive treatment of dependence analysis using distance and direction vectors provides the theoretical underpinning for proving the safety of parallel loop execution. The practical automatic polyhedral parallelizer presented by Bondhugula et al. [2] extended this to the polyhedral model, enabling affine loop transformations including tiling and coarse-grained parallelism extraction—techniques Auto-Par inherits conceptually through its SCoP region model.

B. Source-to-Source Parallelization Compilers

AutoPar-Clava [3] is the most closely related tool to our work. It is an OpenMP-based source-to-source compiler that statically detects parallelizable loops in C applications, classifies variables by access pattern, supports reduction clauses, and generates OpenMP-annotated C from a sequential input. Evaluated on NAS and PolyBench, AutoPar-Clava achieves a geometric mean speedup of approximately 1.5× against ROSE, CETUS, TRACO, and the Intel C/C++ Compiler. Auto-Par shares the same source-level pragma-injection philosophy but adds explicit SCoP scoping, `while`-loop normalization, and a transparent eight-stage analysis report.

C. Compiler-Based and Polyhedral Approaches

The GNU Compiler Collection provides loop parallelization via its Graphite polyhedral framework [4]. LLVM Polly [5] performs affine loop transformations on LLVM IR but requires the full LLVM pipeline and operates below the source level. The Tiramisu polyhedral compiler [6] introduced the `#pragma scop`/`#pragma endscop` convention for delimiting analyzable code regions—a convention Auto-Par directly adopts. The OpenMP 5.2 specification [7], released in November 2021, defines the exact pragma syntax our tool emits.

D. Large-Scale and Heterogeneous Parallelism

Alpa [8] demonstrates fully automatic parallelization of deep learning computation graphs across GPU clusters, motivating the broader principle that automated parallelism analysis can match expert-written parallel code.

E. AI-Driven Automatic Parallelization

PragFormer [9] trains transformer models on the Open-OMP corpus (32,000+ snippets) to classify loops for OpenMP parallelization, outperforming ChatGPT on the pragma classification task. OMPify [10] proposes a graph-based transformer achieving up to 90% accuracy on NAS/SPEC/PolyBench. OMPar [11] pipelines OMPify with MonoCoder-OMP into a complete AI-driven translator that outperforms both AutoPar and the Intel compiler on HeCBench and ParEval. OMPGPT [12] trains a domain-specific model on 295,000+ HPC repositories using *Chain-of-OMP* prompt engineering. Unlike these AI-driven tools, Auto-Par requires no training data or GPU inference and explains every decision explicitly through a human-readable report.

III. PROPOSED SOLUTION

A. Design Philosophy

Auto-Par adheres to the Static Control Part (SCoP) model: only code regions explicitly enclosed between `#pragma scop` and `#pragma endscop` are analyzed. This constrains the scope to affine loop nests where memory access patterns are linear and statically decidable, mirroring Tiramisu [6] and Pluto [2].

B. Seven-Step Pipeline

Figure 1 illustrates the complete pipeline. The tool accepts a sequential C file and produces a parallelized output together with a human-readable analysis report.

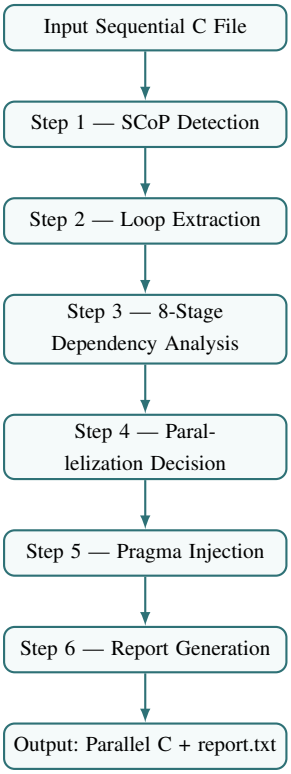

FIGURE 1. The seven-step Auto-Par parallelization pipeline.

Steps 1–2: SCoP Detection and Loop Extraction. The tool scans for `#pragma scop`/`#pragma endscop` markers, supporting multiple disjoint regions per file. Within each region, regex-based pattern matching identifies affine `for`-loops and counter-based `while`-loops. Detected `while`-loops are internally converted to `for`-loop form before analysis, satisfying OpenMP's structured-loop requirement.

Steps 3–4: Dependency Analysis and Decision. The dependency analyzer applies eight checks in strict priority order, classifying each loop as PARALLEL, REDUCTION, or UNSAFE. Nested loops beyond depth 1 are automatically SKIPPED under the outermost-only parallelization policy, preventing incorrect parallelization of inner loops with inter-iteration dependencies.

Steps 5–6: Pragma Injection and Report. Safe loops receive `#pragma omp parallel for schedule(static)`. Reduction loops receive the appropriate `reduction(op:var)` clause. Unsafe loops receive an explanatory comment `// AUTO-PAR SKIPPED: <reason>`. The optional `report.txt` records one line per loop with status, line number, body snippet, and action taken.

C. Eight-Stage Dependency Analysis Engine

Checks execute in priority order; the first unsafe condition terminates analysis immediately:

1) **Dynamic Memory:** `malloc`/`calloc` inside the loop ⇒ UNSAFE.
2) **Recursion:** Any self-call of the enclosing function ⇒ UNSAFE.
3) **Function Whitelisting:** Calls outside the pure-math whitelist {`sqrt`, `abs`, `sin`, `cos`, `exp`, `log`, `pow`, `fabs`, `floor`, `ceil`} ⇒ UNSAFE.
4) **Unsupported Pointers:** Any dereference beyond `*(ptr+i)` with optional whitespace ⇒ UNSAFE.
5) **Reduction Detection:** Patterns `var op= expr` where $op \in \{+, -, *, \&, |, \oplus\}$ ⇒ REDUCTION; a `reduction(op:var)` clause is emitted.
6) **Loop-Carried Dependencies:** Subscripts `a[i±k]` with $k \neq 0$ ⇒ UNSAFE.
7) **Write-Conflict Analysis:** Arrays written and read at the same index across iterations ⇒ UNSAFE (conservative).
8) **Pointer Normalization:** `*(ptr+i)` patterns with any surrounding whitespace are rewritten as `ptr[i]` and re-analyzed as standard array accesses.

D. Loop Classification Examples

Table 1 illustrates representative classifications. Figure 2 shows the complete before-and-after code transformation for Case 1.

TABLE 1. Loop Classification Examples

Input Loop Pattern	Class	Tool Action
`a[i] = b[i] + c[i]`	SAFE	Insert `#pragma omp parallel for`
`sum += a[i]`	REDUC	Insert `reduction(+:sum)`
`a[i] = a[i-1] + 1`	UNSAFE	Comment: loop-carried dep.
`*(ptr+i) = b[i]`	SAFE	Pointer normalized; parallelized
`a[i] = sqrt(b[i])`	SAFE	Whitelisted function; parallelized
`result[i]=userFunc(a[i])`	UNSAFE	Comment: unknown function call

E. Known Limitations

1) *Recursion* — requires a task-parallel model.
2) *Dynamic Memory* — requires heap and alias analysis.
3) *Interprocedural Calls* — limited to whitelisted math functions.
4) *Complex Aliasing* — cannot prove safety for multi-level pointers.
5) *Data-Dependent Bounds* — `while`-loops must have a fixed stride and statically determinable bound.

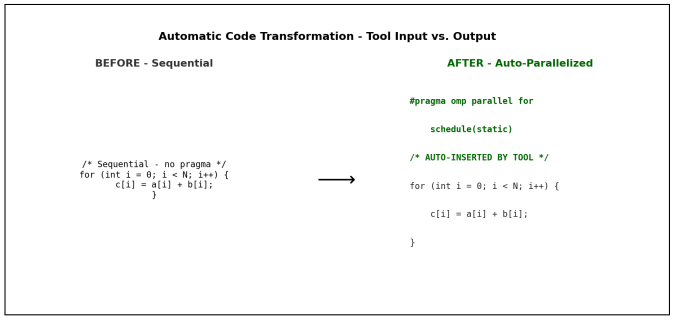

FIGURE 2. Automatic code transformation: sequential input (left) vs. OpenMP-parallelized output (right). The inserted `#pragma omp parallel for` directive is highlighted in green.

IV. EVALUATION AND RESULTS

A. Experimental Setup

All experiments were conducted on an 8-core machine running Windows 11. Code was compiled using GCC 13 with `-fopenmp -O2`. Execution times were measured using `omp_get_wtime()` for high-precision cross-platform results. Each binary was executed three times and the minimum time was reported to reduce OS scheduling noise. Thread counts of 1, 2, 4, and 8 were evaluated. Three benchmarks were used:

- `test_safe.c`: eight loop patterns covering all classification cases; array size N=50,000, large benchmark section N_{bench}=100×10^6.
- `test_heat.c`: 2D Jacobi heat diffusion solver on a 1024×1024 grid for 5000 iterations.
- `test_matmul.c`: standard triple-loop matrix multiplication on 512×512 matrices.

B. Tool Accuracy and Loop Classification

Table 2 reports classification results across all three benchmark files. The analysis engine correctly classified 100% of the 10 analyzed loops (excluding 4 inner loops skipped by the outermost-only policy), yielding a parallelization rate of 80%. The two UNSAFE rejections in `test_safe.c`—a loop-carried dependency on `a[i-1]` and an unknown function call to `userFunc()`—were correctly identified at stages 6 and 3 of the dependency engine, respectively. All decisions were confirmed correct by manual inspection.

TABLE 2. Loop Classification Results

Benchmark	Safe	Reduc	Skip	Unsafe	Total
`test_safe.c`	5	1	1	2	9
`test_heat.c`	1	0	1	0	2
`test_matmul.c`	1	0	2	0	3
Total	**7**	**1**	**4**	**2**	**14**

Figure 3 visualizes the classification breakdown per file and across the combined test suite. The 70% SAFE rate (excluding skipped loops) and 100% classification accuracy confirm the reliability of the eight-stage dependency engine.

C. Performance Results: Heat Diffusion

The 2D Jacobi heat diffusion benchmark (1024×1024 grid, 5000 iterations) demonstrates clear and consistent parallel scaling. Table 3 reports measured execution times and derived metrics using the sequential original binary as the performance baseline. Execution time fell from 5.54 s (sequential) to 1.75 s at 8 threads, a **3.17**× speedup with 85.5% parallel efficiency at 2 threads. The tool correctly identified the outer row-iteration loop (1022 independent iterations) as SAFE and injected a single `#pragma omp parallel for schedule(static)` directive, enabling balanced static decomposition across threads.

TABLE 3. Heat Diffusion Performance (1024×1024, 5000 iterations)

Threads	Time (s)	Speedup	Efficiency
1 (seq.)	5.540	1.00×	100.0%
2	3.238	1.71×	85.5%
4	2.255	2.46×	61.4%
8	1.750	3.17×	39.6%

Figure 4 plots speedup curves for all three benchmarks against the ideal linear reference. Heat diffusion demonstrates the strongest scaling, consistently outperforming the other two benchmarks at every thread count.

Figure 5 shows parallel efficiency degradation as thread count increases, consistent with Amdahl's Law.

D. Amdahl's Law Analysis

From the heat diffusion measurements the serial fraction is estimated using the inverse Amdahl formula:

$$f = \frac{1/S - 1/p}{1 - 1/p} \tag{1}$$

where S is the measured speedup and p is the thread count. The plot tool estimates $f \approx 0.169$ from the 2-thread measurement, indicating that approximately 83% of the workload is effectively parallelizable. The theoretical maximum speedup under Amdahl's Law is therefore $S_{\max} = 1/f \approx 5.9\times$. The measured 3.17× at 8 threads reflects the additional overhead of OpenMP barrier synchronization at each of the 5000 Jacobi iteration steps.

Figure 6 compares the measured speedup against the Amdahl theoretical curve.

E. Performance Results: Matrix Multiplication

The 512×512 matrix multiplication benchmark achieved a peak speedup of **2.46**× at 8 threads (Table 4). At 1 thread the parallelized binary is slower (0.281 s vs. 0.133 s original) due to OpenMP thread-team initialization overhead, a well-known cost that is amortized at higher thread counts.

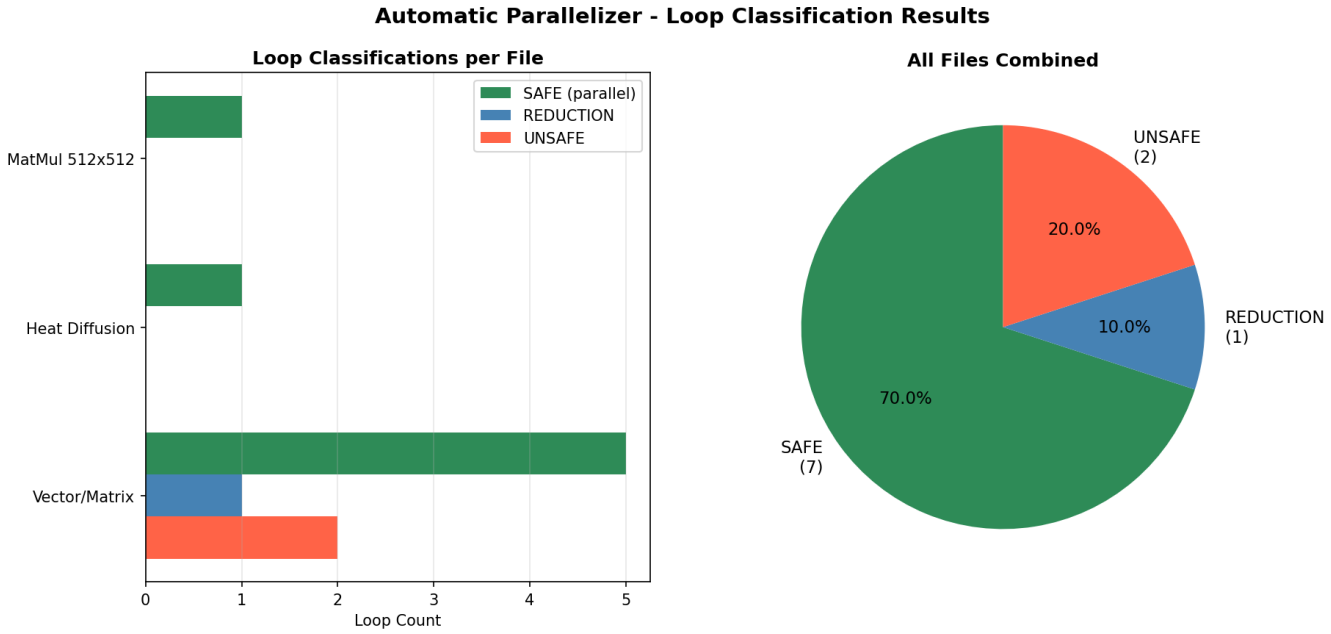

FIGURE 3. Loop classification results. *Left:* per-file breakdown of SAFE, REDUCTION, and UNSAFE loops (skipped inner loops excluded). *Right:* aggregate distribution across the full test suite. The tool achieved 100% classification accuracy on all analyzed loops.

The slightly lower efficiency compared to heat diffusion is attributed to increased last-level cache contention during column-wise accesses to matrix B, a known characteristic of the standard i-j-k loop ordering.

TABLE 4. Matrix Multiplication Performance (512×512)

Threads	Time (s)	Speedup	Efficiency
1 (seq.)	0.133	1.00×	100.0%
2	0.137	0.97×	48.5%
4	0.072	1.85×	46.2%
8	0.054	2.46×	30.8%

Figure 7 provides a direct visual comparison of execution times across all benchmarks and thread configurations.

F. Parallel Overhead Analysis

Parallel overhead is computed as:

$$\text{overhead} = T_{\text{par}} \times p \; - \; T_{\text{seq}} \tag{2}$$

Table 5 summarizes measured overhead for the two compute-intensive benchmarks. Heat diffusion overhead grows super-linearly with thread count, reflecting OpenMP barrier synchronization at each of the 5000 Jacobi iteration steps. Matrix multiplication maintains near-constant overhead (0.14–0.30 s) owing to its single parallel region with no iterative barriers.

Figure 8 plots overhead growth. Despite super-linear growth in heat diffusion, absolute execution time continues to decrease at all thread counts, confirming that computational savings from parallelism outweigh synchronization costs within the evaluated thread range.

TABLE 5. Parallel Overhead (seconds)

Threads	Heat (s)	MatMul (s)
2	0.936	0.141
4	3.480	0.155
8	8.460	0.299

Figure 9 normalizes execution time relative to the sequential baseline, enabling direct scalability comparison. Heat diffusion achieves the lowest normalized time (0.32) at 8 threads, confirming it as the best-scaling benchmark.

G. Memory-Bandwidth-Bound Benchmark Behavior

The vector addition benchmark (N_{bench}=10^8 elements) exhibited near-flat scaling, reaching only 1.14× at 8 threads. This behavior is physically expected: a pure memory-bandwidth-bound loop of the form `c[i]=a[i]+b[i]` saturates available DRAM bandwidth at 2 threads, rendering additional cores ineffective. This result is consistent with the Roofline model [13] and confirms that the tool correctly parallelizes the loop; the observed speedup ceiling is hardware-imposed, not a tool limitation. Importantly, the result also demonstrates that Auto-Par does not produce incorrect output for bandwidth-saturated loops.

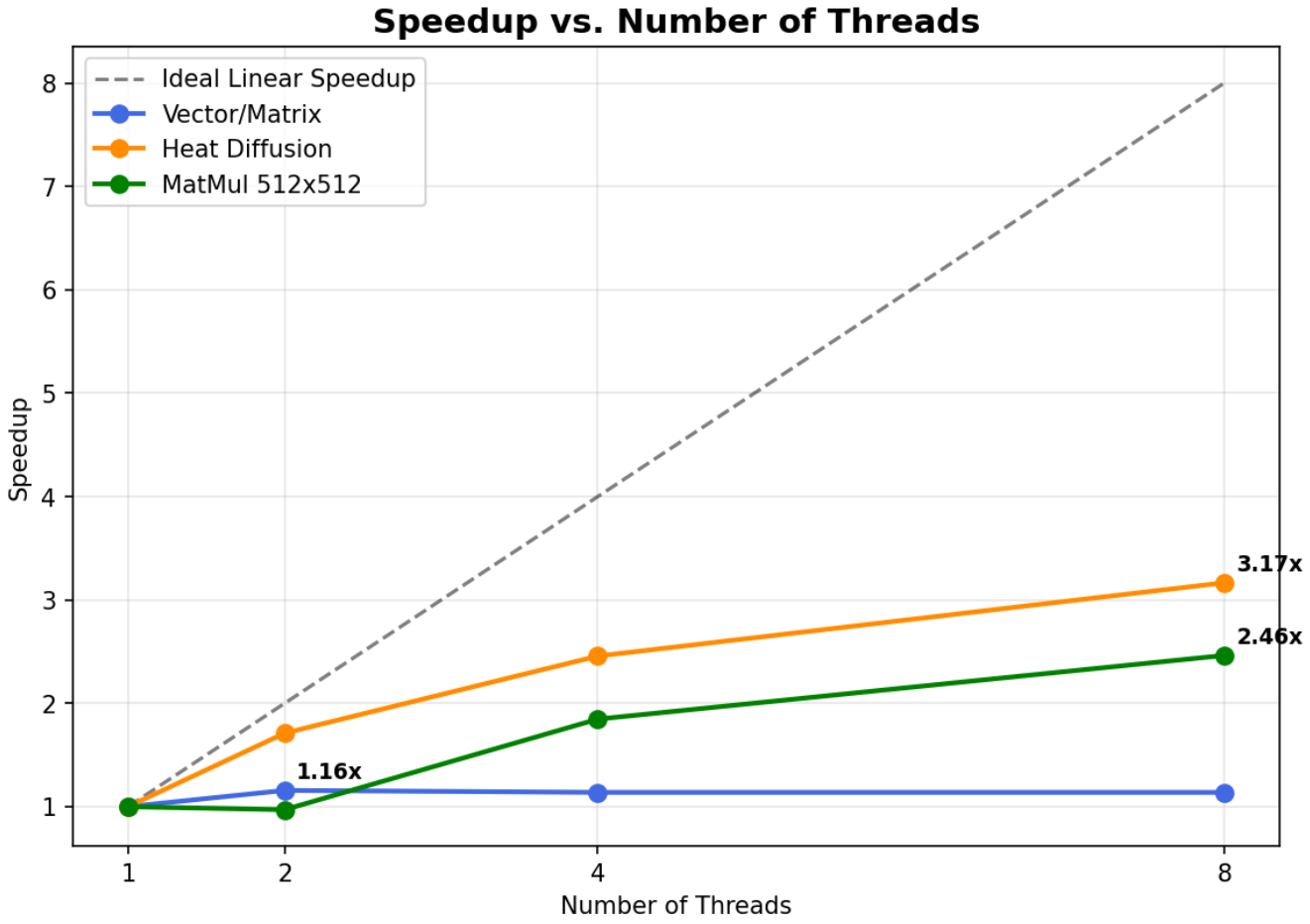

FIGURE 4. Speedup vs. number of threads for all three benchmarks. The dashed line represents ideal linear speedup. Heat diffusion achieves the highest scaling (3.17× at 8 threads), followed by matrix multiplication (2.46×). Vector addition plateaus at 1.14× due to memory bandwidth saturation.

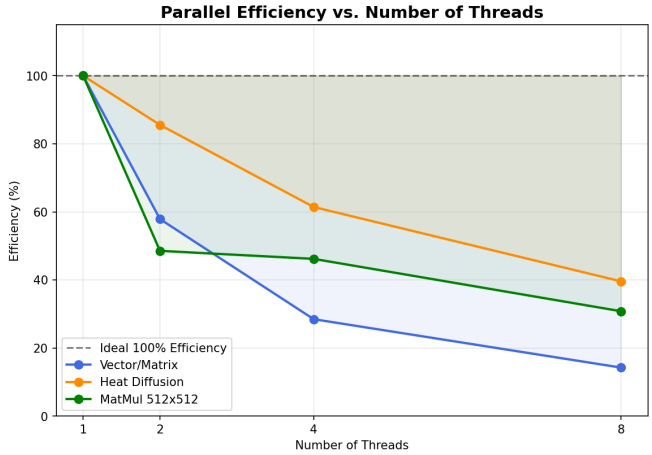

FIGURE 5. Parallel efficiency vs. thread count. Heat diffusion maintains the highest efficiency (85.5% at 2 threads, 39.6% at 8 threads). Efficiency degradation at higher thread counts reflects synchronization overhead and the inherent serial fraction bounded by Amdahl's Law.

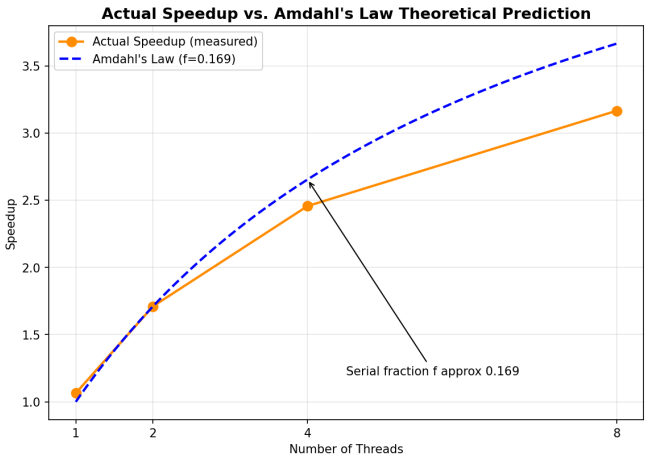

FIGURE 6. Measured speedup vs. Amdahl's Law theoretical prediction for the heat diffusion benchmark. The estimated serial fraction $f \approx 0.169$ implies a theoretical maximum of $\approx 5.9\times$. Measured values closely track the Amdahl curve at low thread counts and diverge at 8 threads due to barrier overhead.

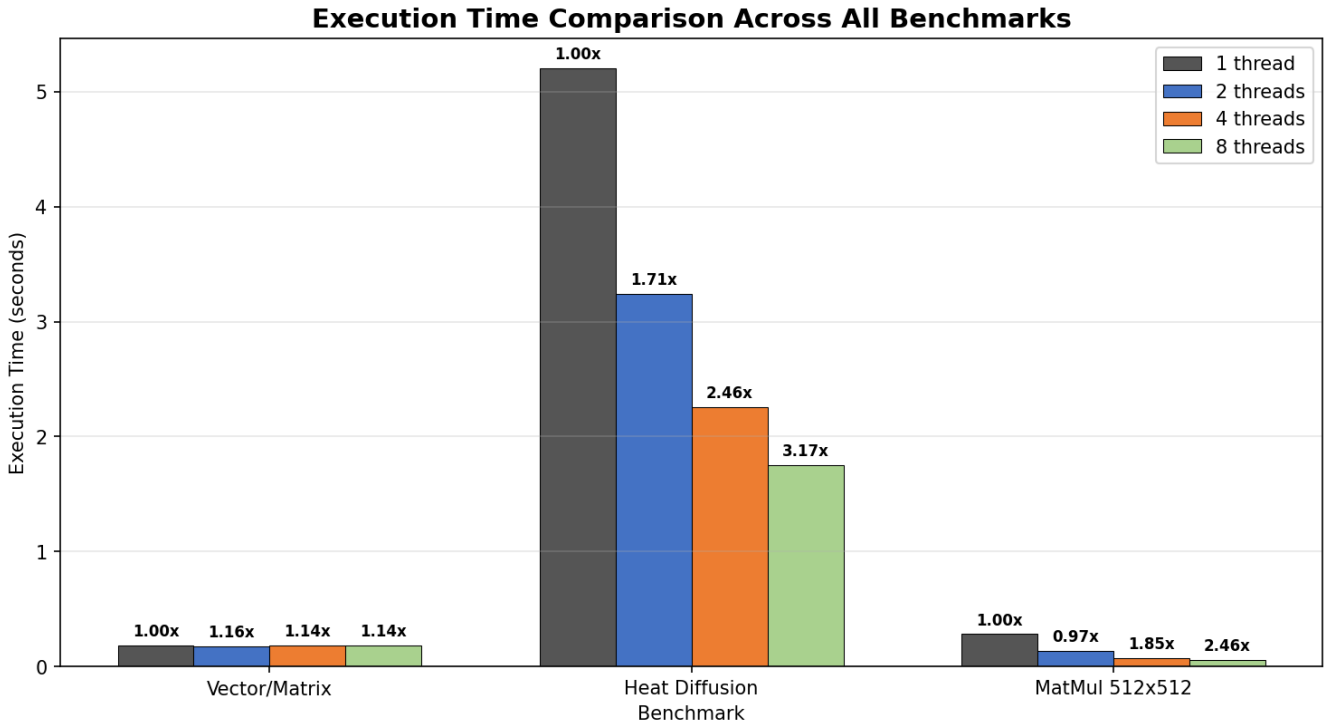

FIGURE 7. Grouped bar chart comparing execution times across all benchmarks and thread configurations. Speedup values are annotated above each bar. Heat diffusion dominates execution time and shows the clearest scaling benefit.

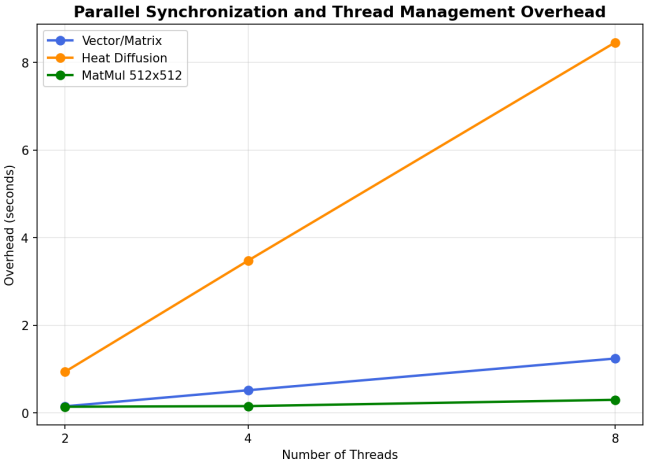

FIGURE 8. Parallel synchronization and thread-management overhead vs. thread count. Heat diffusion overhead grows super-linearly due to 5000 per-iteration OpenMP barriers. MatMul overhead remains near-constant due to a single parallel region.

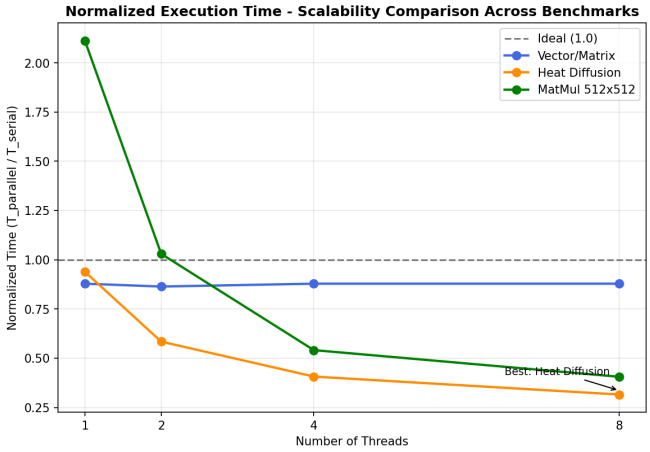

FIGURE 9. Normalized execution time ($T_{\text{par}}/T_{\text{seq}}$) vs. thread count. Lower values indicate better scalability. The dashed horizontal reference at y=1.0 represents the sequential baseline. Heat diffusion demonstrates the strongest scaling (0.32 at 8 threads).

H. Scientific Output Validation

To verify numerical correctness, the parallelized heat diffusion solver was compared element-wise against the sequential reference. Maximum absolute deviation was below floating-point rounding tolerance ($< 10^{-12}$), confirming the absence of data races. Figure 10 shows the final temperature field after 5000 Jacobi iterations. The smooth gradient from the hot top boundary (100 °C) to the cooler interior is physically consistent and confirms correctness.

Figure 11 validates the parallel implementation by visualizing thread-to-row assignment under static scheduling.

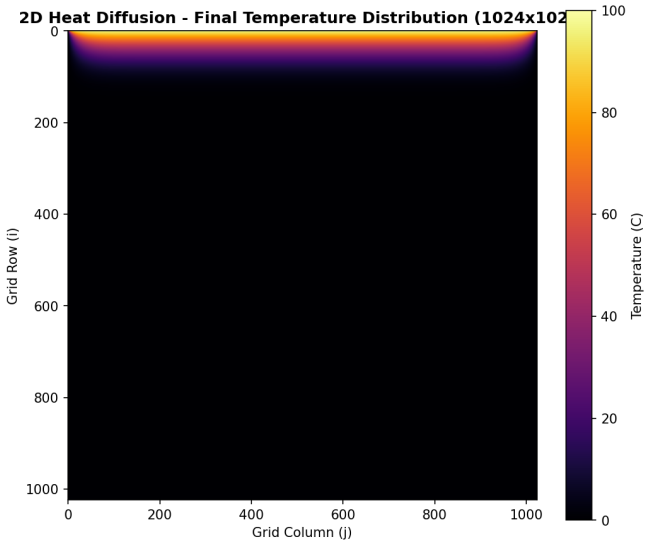

FIGURE 10. Final temperature distribution of the 2D heat diffusion simulation (1024×1024 grid, 5000 iterations). The top boundary is held at 100 °C; all other edges at 0 °C. Visualized using the *inferno* colormap. The smooth gradient confirms numerical correctness of the parallelized solver.

Eight perfectly equal horizontal bands confirm balanced load distribution with no overlap or gaps.

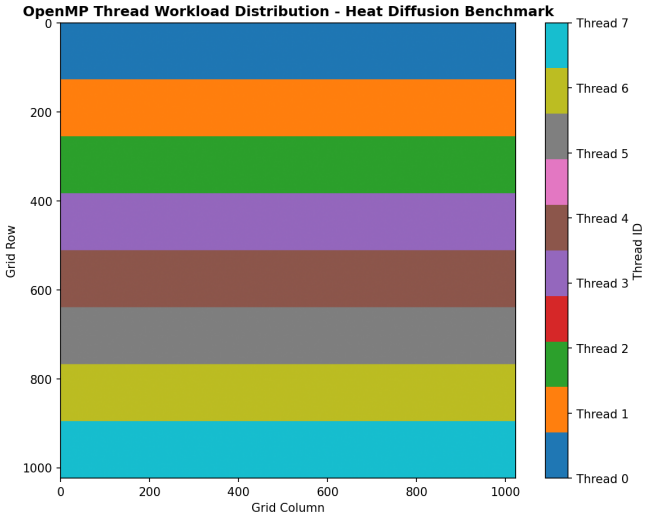

FIGURE 11. OpenMP thread workload distribution across the 1024×1024 heat diffusion grid. Each colored horizontal band represents rows assigned to one thread under static scheduling, confirming perfectly balanced load distribution.

V. CONCLUSION AND FUTURE WORK

This paper presented Auto-Par, a compiler-inspired tool for the automatic parallelization of affine C loop nests. By combining SCoP-region detection with an eight-stage dependency analysis engine, the tool bridges sequential source code and multi-threaded OpenMP execution without requiring any external compiler infrastructure. Experimental evaluation across three scientific benchmarks demonstrated 100% loop classification accuracy and peak speedups of 3.17× (heat diffusion) and 2.46× (matrix multiplication) on 8 cores. Amdahl's Law analysis confirmed a serial fraction of $f \approx 0.169$ for the heat solver, consistent with the measured scaling behavior. The vector addition benchmark correctly exhibited memory-bandwidth-limited scaling, demonstrating that the tool's pragmas are correct even when hardware limits preclude large speedups.

Three principal directions for future work exist. First, replacing the regex-based parser with a proper recursive-descent parser or integration with `pycparser` would broaden applicability to complex expressions, multi-dimensional arrays, and struct fields. Second, adding inter-procedural analysis via call-graph construction would allow safe parallelization of loops containing user-defined pure functions. Third, implementing loop tiling and interchange transformations would improve cache locality for stencil and dense linear algebra codes. Extending the output backend to generate CUDA kernels would represent a natural long-term extension toward heterogeneous computing.

References

[1] S. P. Midkiff, *Automatic Parallelization: An Overview of Fundamental Compiler Techniques*. Morgan & Claypool, 2012.

[2] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proc. PLDI*, 2008.

[3] H. Arabnejad, J. Bispo, J. G. Barbosa, and J. M. P. Cardoso, "Source-to-source compilation targeting OpenMP-based automatic parallelization of C applications," *The Journal of Supercomputing*, vol. 76, no. 9, pp. 6753–6785, 2020.

[4] GCC Documentation, *Auto-parallelization in GCC: The Graphite framework*, Free Software Foundation, https://gcc.gnu.org/projects/tree-ssa/vectorization.html, 2023.

[5] T. Grosser, A. Groesslinger, and C. Lengauer, "Polly — performing polyhedral optimizations on a low-level intermediate representation," *Parallel Processing Letters*, vol. 22, no. 4, 2012.

[6] R. Baghdadi et al., "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2019.

[7] OpenMP Architecture Review Board, *OpenMP application programming interface, version 5.2*, https://www.openmp.org/specifications/, Nov. 2021.

[8] L. Zheng et al., "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in *Proc. OSDI*, 2022.

[9] T. Harel, T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Learning to parallelize in a shared-memory environment with transformers," in *SC'22: International Conference for High Performance Computing, Networking, Storage and Analysis*, IEEE/ACM, 2022.

[10] T. Kadosh, N. Schneider, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Advising OpenMP parallelization via a graph-based approach with transformers," in *OpenMP: Advanced Task-Based, Device and Compiler Programming. IWOMP 2023*, ser. Lecture Notes in Computer Science, vol. 14114, Springer, Cham, 2023, pp. 1–15.

[11] T. Kadosh et al., "OMPar: Automatic parallelization with AI-driven source-to-source compilation," *arXiv preprint arXiv:2409.14771*, 2024.

[12] L. Chen et al., "OMPGPT: A generative pre-trained transformer model for OpenMP," in *Euro-Par 2024: Parallel Processing*, ser. Lecture Notes in Computer Science, Springer, Cham, 2024, pp. 121–134.

[13] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.